PHASE 3 FREEZE HANDOVER PACKAGE

DeepMindQ Sales Intelligence Platform

Comprehensive architecture, governance, and technical handover documentation covering the AI-governed research pipeline, evidence-grounded intelligence, RFP/RFI detection foundation, and human-controlled selling architecture.

PLATFORM
DeepMindQ CRM

INFRASTRUCTURE
Vercel + Neon PostgreSQL + Prisma v6.19

DATE
July 19, 2026

Table of Contents

1. Executive Summary

This document constitutes the official Phase 3 freeze handover package for the DeepMindQ AI-Governed Sales Intelligence Platform. Phase 3 represents the intelligence hardening milestone where the platform transitioned from a basic CRM with AI assistance to a fully governed, evidence-grounded intelligence system. Every LLM call now passes through a mandatory governance layer that enforces confidence thresholds, hallucination prevention, evidence traceability, and comprehensive audit logging. The platform is deployed on Vercel with Neon PostgreSQL as its persistent data store, and the database schema has been validated and synchronized via `npx prisma db push`.

The architecture comprises 152 API route handlers organized across 8 domain groups, a 6-step research intelligence pipeline, a 4-factor weighted signal-to-capability matching engine, and a dual-function AI governance gateway. The Prisma schema defines 35 models with Phase 3 additions including AIGenerationAudit (11 audit fields), Evidence (7 indexes), and SignalCapabilityMatch (6 indexes). Human-controlled selling is enforced by design: no cron jobs are configured, the email worker only processes human-approved drafts from the SendQueue, and all AI outputs are advisory drafts requiring explicit human approval before any customer communication. A known design tension exists in sequences__process.ts which auto-approves sequence drafts, and this is flagged as a mandatory Phase 4 architectural control.

This handover package covers: system architecture with diagrams, data flow documentation, module dependency analysis, AI governance rules and thresholds, API entry point contracts, database schema summary, RFP/RFI intelligence foundation, human-controlled selling architecture, known technical debt inventory, and Phase 4/5/6 dependency rules. One explicit Phase 4 requirement has been documented: the sequence automation architecture must permanently enforce human approval before any customer communication, and no cron, worker, scheduler, or automation may ever bypass human review.

2. System Architecture

The DeepMindQ platform follows a layered architecture with clear separation between client, API routing, core business logic, and infrastructure. The frontend is a Next.js 18 application using React with Tailwind CSS and the shadcn/ui component library. All server-side logic resides in Next.js API route handlers that delegate to core library modules. The AI governance layer sits between the AI-consuming modules and the raw LLM provider chain, acting as a mandatory gateway for every AI generation request. This section presents the high-level architecture, the technology stack, and the deployment topology.

2.1 Architecture Overview

The diagram below illustrates the four-layer architecture: Client Layer, API Layer (152 routes across 8 domain groups), Core Libraries (including the mandatory AI Governance Gateway), and Infrastructure (Prisma ORM, Neon PostgreSQL, Vercel Edge Runtime, and external AI/search APIs). The governance layer is visually distinguished as the mandatory gate through which all AI requests must pass before reaching any LLM provider.

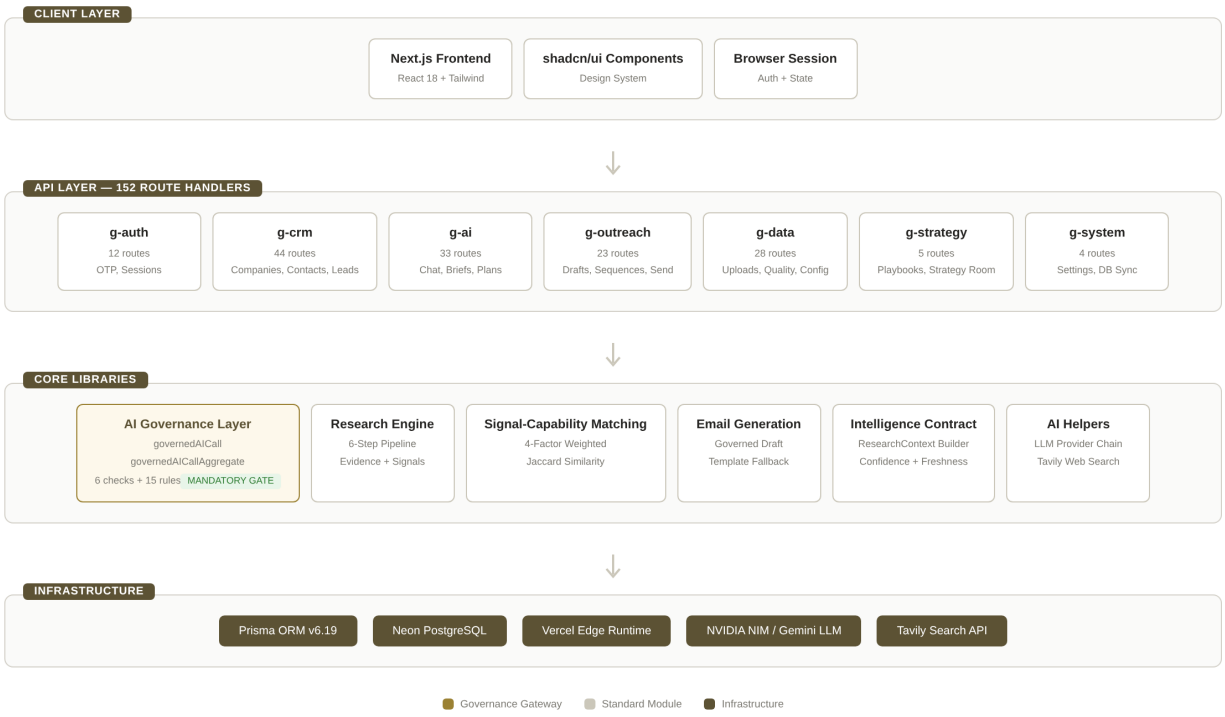

Figure 1: System Architecture Overview

2.2 Technology Stack

Layer	Technology	Version / Detail
Frontend	Next.js + React	App Router, Server Components
UI	shadcn/ui + Tailwind CSS	Component library + utility CSS
API	Next.js Route Handlers	152 handlers across 8 groups
ORM	Prisma	v6.19.3, PostgreSQL provider
Database	Neon PostgreSQL	Serverless, pooler connection
Hosting	Vercel	Edge Runtime, project prj_JtenBM...
LLM	NVIDIA NIM, Gemini, Fireworks	Multi-provider chain with fallback
Search	Tavily API	Advanced search depth, AI answer
Auth	OTP-based + Session tokens	Custom implementation

Table 1: Technology Stack

2.3 Deployment Architecture

The platform is deployed entirely on Vercel under team `team_nKBskve4kFghKK8BNVNe0b2Q` with project ID `prj_JtenBMINFkDNVYw7pbVXVUUTe7iK`. The Neon PostgreSQL database runs in the us-east-1 AWS region with connection pooling enabled via the pooler endpoint. The `vercel.json`

configuration file is intentionally empty (no custom rewrites, redirects, headers, or cron jobs), confirming that the platform relies on Vercel defaults for routing and has zero scheduled background processes. All API routes use the catch-all slug pattern `[...slug]` for domain grouping, which maps file paths to URL segments via the Next.js file-system router. Environment variables including `DATABASE_URL` and AI provider API keys are managed through the Vercel dashboard and are not committed to source control.

3. Data Flow Architecture

The core intelligence pipeline follows a 6-step process that transforms raw web search results into governed, evidence-grounded AI outputs. This pipeline is the backbone of the platform and represents the primary value chain: from company creation through research, evidence collection, signal detection, capability matching, and finally AI-assisted content generation with full audit traceability. Each step is designed to produce intermediate artifacts that serve both immediate processing needs and downstream analytical queries.

3.1 Research Intelligence Pipeline

The research pipeline, implemented in `src/lib/research-engine/researcher.ts`, executes six sequential steps. Step 1 fires four parallel Tavily web searches targeting business overview, technology landscape, key people, and recent news. Step 2 stores each search result as an Evidence record with source URL, relevance score, snippet text, and quality tier classification. Step 3 uses `governedAICallAggregate` at two points (steps 3a and 3c) to extract structured data: main company data, key people, and the Phase 3 enhanced fields including structuredTechLandscape, strategicPriorities, businessProblems, transformationAreas, and technologyThemes. Step 4 cross-references extracted data against evidence records and computes per-field confidence scores. Step 5 persists the complete intelligence package to the database. Step 6 triggers downstream AI generation through the governance gateway.

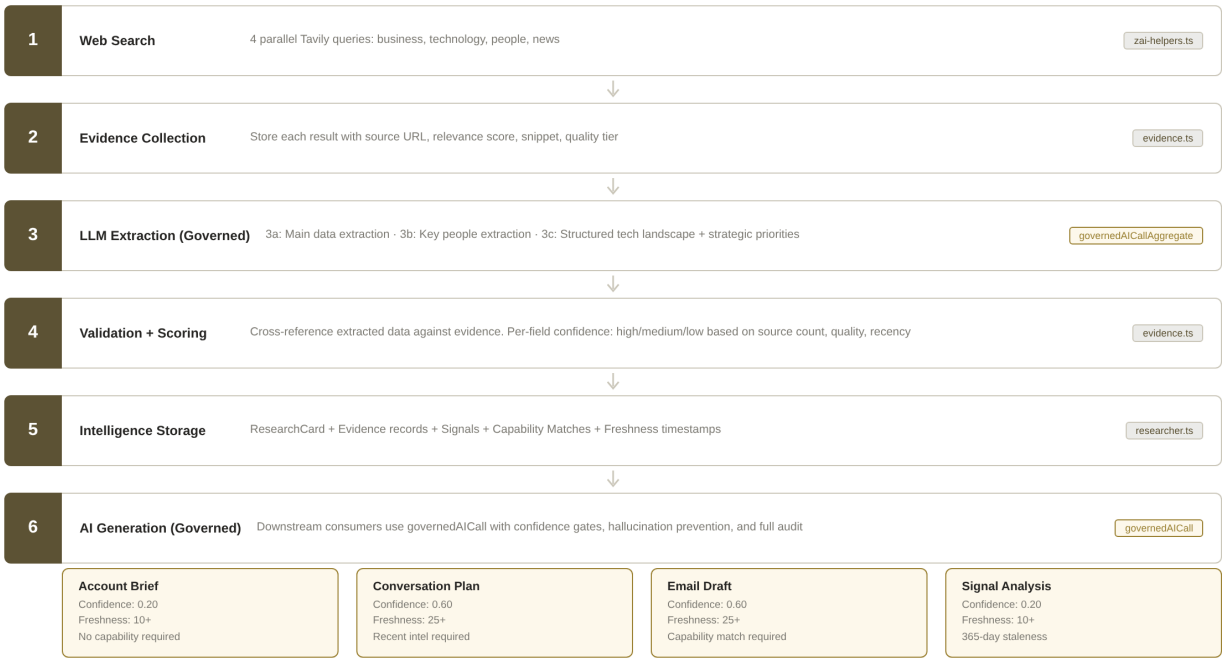

Figure 2: Research Intelligence Pipeline (6 Steps)

3.2 Signal Detection and RFP/RFI Intelligence

Signal detection (`src/lib/research-engine/signals.ts`) uses `governedAICallAggregate` to identify buying signals from research context. The LLM prompt includes specific RFP/RFI/tender fields: `opportunityType` (rfp, rfi, tender, vendor_search, procurement, tech_transformation), `publicationDate`, `deadline`, `buyingArea`, `techRequirement`, `serviceRequirement`, and `matchingCapability`. Each detected signal is stored in the CompanySignal model with an `evidenceIds` JSON array linking back to source evidence records. The CompanySignal model includes 8 database indexes including composite indexes on (companyId, signalType, createdAt) and (companyId, status) for efficient querying.

3.3 Signal-to-Capability Matching

The matching engine (`src/lib/research-engine/signal-capability-matching.ts`) scores each signal against the capability knowledge base using a 4-factor weighted formula. Category matching contributes 30% of the score, keyword overlap (Jaccard similarity) contributes 30%, business problem alignment contributes 20%, and an impact bonus contributes 5%. The `SIGNAL_CAPABILITY_MAP` defines signal-type-to-category mappings for 8 signal types: funding_round, acquisition, tech_stack_change, expansion, partnership, regulatory, and financial_pressure. Each mapping includes target capability categories, typical business problems, suggested sales angles, and keyword lists. The minimum match score threshold is 0.25, and an LLM-enhancement fallback activates for signals scoring below 0.35. Match results produce a `reason`, `businessProblem`, `salesAngle`, and `expectedOutcome` for each match, stored in the SignalCapabilityMatch model.

4. Module Dependency Map

The platform modules follow a strict dependency hierarchy designed to prevent governance bypass. At the foundation, `zai-helpers.ts` provides raw LLM provider chain management and Tavily web search. The AI governance layer (`ai-governance.ts`) is the only authorized consumer of `callLLM()` from zai-helpers. All other modules that need AI capabilities must import from `ai-governance.ts`, never directly from `zai-helpers.ts`. This architectural constraint is enforced by code comments, a build-time guard script, and the dependency structure itself. The intelligence-contract module (`intelligence-contract.ts`) provides a unified `ResearchContext` type that assembles data from the research card, evidence records, signals, and capability matches into a single object consumed by all governed AI calls.

Module	Depends On	Consumed By
zai-helpers.ts	ai-config.ts, External APIs	ai-governance.ts ONLY
ai-governance.ts	zai-helpers.ts (callLLM), db	All AI route handlers
intelligence-contract.ts	db (Prisma)	ai-governance.ts, AI routes
research-engine/researcher.ts	zai-helpers, ai-governance, evidence, signals	API: companies__research
research-engine/evidence.ts	db, zai-helpers (types)	researcher.ts
research-engine/signals.ts	ai-governance, zai-helpers	researcher.ts
research-engine/signal-capability-matching.ts	db	researcher.ts (step 6)
email-generation.ts	ai-governance, intelligence-contract, db	API: contacts__generate-email, sequences__process
ai__chat.ts	ai-governance (governedAICallAggregate), db	Frontend chat component
ai__account-brief.ts	ai-governance (governedAICall), intelligence-contract	Frontend company page
ai__conversation-plan.ts	ai-governance (governedAICall), intelligence-contract	Frontend company page

Table 2: Module Dependency Map

5. AI Governance Architecture

The AI governance layer (`src/lib/ai-governance.ts`, approximately 1085 lines) is the single mandatory gateway for all LLM calls in the application. It provides five core capabilities: confidence gates with per-generation-type thresholds, six pre-generation governance checks, fifteen hallucination prevention

rules injected into every prompt, evidence-grounded context injection, and comprehensive audit logging. The governance layer uses a non-throwing design where checks return structured results rather than throwing exceptions, allowing AI route handlers to make contextual decisions about how to handle governance failures.

5.1 Governance Checks

Six pre-generation checks are evaluated before any company-specific LLM call. The research_exists check verifies that a CompanyResearchCard record exists for the target company. The research_confidence check compares the average per-field confidence score against the minimum threshold for the generation type. The freshness_score check validates that the composite freshness metric (0-100) meets the minimum requirement. The staleness check calculates days since the last research update and compares against the maximum allowed staleness period. The capability_match check verifies that at least one capability asset matched the company signals (when required by the generation type config). The recent_intelligence check ensures that recent signals or evidence exist for the company.

5.2 Confidence Thresholds by Generation Type

Generation Type	Min Confidence	Min Freshness	Require Capability	Require Intel	Max Staleness (days)
email_draft	0.60	25	Yes	Yes	60
conversation_plan	0.60	25	No	Yes	60
account_brief	0.20	10	No	No	180
signal_analysis	0.20	10	No	No	365
recommendations	0.40	15	No	No	90
opportunities	0.50	20	No	Yes	90
suggested_contacts	0.30	15	No	No	90

Table 3: Confidence Thresholds by Generation Type

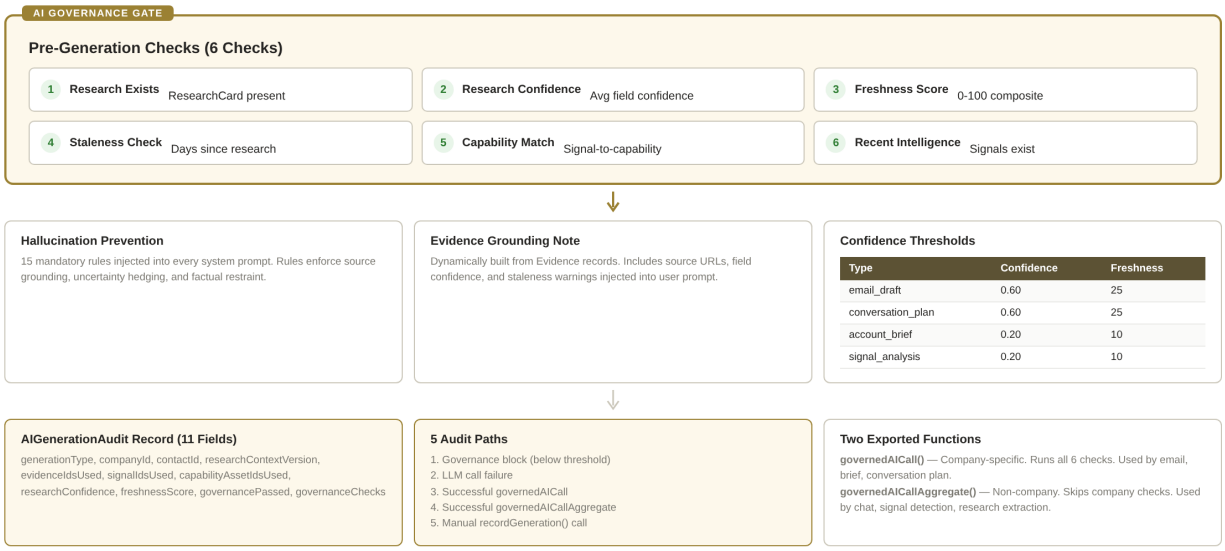

Figure 3: AI Governance Architecture

5.3 Hallucination Prevention

Fifteen hallucination prevention rules are defined as the `HALLUCINATION_PREVENTION_RULES` constant and are injected into the system prompt of every governed LLM call. These rules enforce source-grounded responses, prohibit fabrication of facts not present in the provided context, require uncertainty hedging when confidence is low, mandate citation of specific data points, and prevent the AI from making claims about people, financial figures, or timelines without explicit evidence. The rules also include specific constraints for the sales context: never invent customer names, never fabricate case study metrics, and never claim capabilities that are not present in the capability knowledge base. Additionally, a staleness warning modifier is applied when intelligence data exceeds a certain age, instructing the LLM to be more conservative in its claims and explicitly note the potential for outdated information.

5.4 Audit Trail

Every AI generation, whether successful, blocked, or failed, produces an AIGenerationAudit record with 11 fields: generationType, companyId, contactId, researchContextVersion, evidenceIdsUsed (JSON array), signalIdsUsed (JSON array), capabilityAssetIdsUsed (JSON array), researchConfidence (computed average), freshnessScore (0-100), governancePassed (boolean), and governanceChecks (JSON object with per-check detail). The `recordGeneration()` function is fire-and-forget: errors are logged to console but never thrown, ensuring that audit failures cannot break the user-facing flow. Five distinct audit paths exist: (1) governance block when confidence falls below threshold, (2) LLM call failure with error message, (3) successful `governedAICall` with full context, (4) successful `governedAICallAggregate` for non-company calls, and (5) manual `recordGeneration()` invocations from modules that perform additional validation before calling the governance layer.

5.5 Dual Function Exports

The governance layer exports exactly two functions for external use. `governedAICall()` is designed for company-specific AI generation (email drafts, account briefs, conversation plans). It accepts a ResearchContext object, runs all six governance checks, applies confidence thresholds, injects hallucination prevention rules and evidence grounding notes, calls the LLM, and records the full audit trail. `governedAICallAggregate()` is designed for non-company AI calls (chat, signal detection, research extraction). It skips company-specific checks since no company context exists, but still injects hallucination prevention rules and creates audit records. Both functions return a GovernedAIResult object containing success status, response text, governance result, rejection reason, and the grounding note used in the prompt.

6. API Entry Points

The platform exposes 152 API route handlers organized into 8 domain groups using Next.js catch-all slug routing (`[...slug]`). Each group corresponds to a functional domain with its own subdirectory under `src/app/api/`. The following table summarizes the route groups, their handler counts, and primary responsibilities.

Route Group	Handlers	Primary Responsibility
g-auth	12	OTP login/logout, sessions, password management, profile updates
g-crm	44	Companies, contacts, leads, segments, signals, evidence, pipeline, batches, duplicates, scoring
g-ai	33	Chat, account briefs, conversation plans, knowledge, capabilities, research agent, scoring, insights
g-outreach	23	Drafts, email worker, sequences, templates, tracking, webhooks, queue, unsubscribe
g-data	28	File uploads, data quality, column rules, normalization, scoring config, audit, analytics
g-strategy	5	Playbooks (CRUD), strategy room (account strategy)
g-system	4	Settings, database sync, seed data
cron	1	Job processor (manual trigger only, no cron configured)

Table 4: API Route Groups

6.1 AI Generation Endpoints (Governed)

Four AI endpoints consume the governance layer for company-specific generation. Each endpoint calls `governedAICall()` with a specific generationType, passes the ResearchContext assembled by the intelligence contract, and handles governance results appropriately. The chat endpoint uses `governedAICallAggregate()` since it may operate without company context. The account brief endpoint uses `enforceGovernance: false` to allow generation even with low confidence, producing a best-effort brief with appropriate caveats rather than blocking the user entirely.

Endpoint	Function	Governance Type	Threshold
api/g-ai/ai__chat	General CRM assistant chat	governedAICallAggregate	N/A (non-company)
api/g-ai/ai__account-brief	Account intelligence brief	governedAICall	0.20 confidence (enforce: false)
api/g-ai/ai__conversation-plan	Executive conversation plan	governedAICall	0.60 confidence
lib/email-generation	Personalized email draft	governedAICall	0.60 confidence (enforce: false)
research-engine/signals	Buying signal detection	governedAICallAggregate	N/A (non-company)
research-engine/researcher	Data extraction (steps 3a, 3c)	governedAICallAggregate	N/A (non-company)

Table 5: AI Generation Endpoints and Governance Configuration

7. Database Schema Summary

The Prisma schema (`prisma/schema.prisma`, 1077 lines) defines 35 models organized into functional domains. The database uses PostgreSQL via the Neon serverless provider with Prisma ORM v6.19.3. The schema has been synchronized with the database via `npx prisma db push` and is confirmed to be in sync. This section summarizes the key models with emphasis on Phase 3 additions and the indexes that support query performance.

7.1 Core Domain Models

Model	Purpose	Key Fields	Indexes
Company	Target accounts	industry, domain, intelligenceScore, lifecycleStage	7 indexes
Contact	Prospect contacts	email, leadScore, enrichmentScore, aiConversionScore	7 indexes
CompanyResearchCard	Research intelligence	structuredTechLandscape, strategicPriorities, businessProblems, fieldConfidence	1 (companyId unique)
CompanySignal	Buying signals	opportunityType, publicationDate, deadline, buyingArea, techRequirement, matchingCapability	8 indexes incl. composite
Evidence	Source-grounded facts	extractedField, extractedValue, relevanceScore, confidence, sourceQualityTier	8 indexes incl. composite

Model	Purpose	Key Fields	Indexes
AIGenerationAudit	AI generation trail	11 audit fields: generationType, researchConfidence, freshnessScore, governancePassed, etc.	7 indexes incl. composite
SignalCapabilityMatch	Signal-capability links	matchScore, reason, businessProblem, salesAngle, expectedOutcome	6 indexes incl. composite
CapabilityAsset	Knowledge base	category, keywords, businessProblem, customerOutcome, differentiator	7 indexes

Table 6: Core Domain Models (Phase 3 Emphasis)

7.2 Outreach and Sequence Models

The outreach domain includes Draft, SendQueue, EmailSequence, SequenceStep, and SequenceEnrollment. The Draft model tracks email composition with fields for confidenceScore, sourceSnippetsUsed, assumptionFlags, and reviewNotes. The SendQueue model is the critical control point: emails can only be sent from this queue, and items are only added when a human explicitly approves a draft or when a sequence auto-approves (the design tension flagged in technical debt). The EmailEvent model captures opens, clicks, replies, bounces, and unsubscribes for engagement tracking. The ABTest model supports A/B testing of email variants with draft-level tracking.

7.3 Phase 3 Schema Additions

Phase 3 added significant schema capabilities. The CompanyResearchCard gained structuredTechLandscape (JSON with cloud, data, ai, applications arrays), strategicPriorities (JSON with priority, description, evidence, confidence), businessProblems, transformationAreas, technologyThemes, and four category-specific freshness timestamps (profileFreshnessAt, signalFreshnessAt, contactFreshnessAt, techFreshnessAt). The CompanySignal model gained RFP/RFI intelligence fields: opportunityType, publicationDate, deadline, buyingArea, techRequirement, serviceRequirement, matchingCapability, and sourceQuality. The AIGenerationAudit model was added entirely in Phase 3 with 11 fields and 7 indexes. The SignalCapabilityMatch model was added to store the output of the 4-factor matching engine. The Evidence model was added with 7 indexes including composite indexes for efficient field-level and company-level evidence queries.

8. RFP/RFI Intelligence Foundation

Phase 3 established the data architecture foundation for RFP (Request for Proposal) and RFI (Request for Information) intelligence. This foundation enables the platform to detect procurement opportunities, match them against organizational capabilities, and provide sales teams with structured, evidence-grounded intelligence for competitive bidding. The RFP/RFI capability is not a standalone module but rather a cross-cutting concern embedded in the signal detection, capability matching, and AI generation layers.

8.1 Signal-Level RFP/RFI Fields

When the signal detection engine identifies a procurement-related signal, it populates dedicated RFP/RFI fields on the CompanySignal model. The `opportunityType` field classifies the signal as rfp, rfi, tender, vendor_search, procurement, tech_transformation, or partner_requirement. The `publicationDate` and `deadline` fields capture the procurement timeline. The `buyingArea` field describes the procurement category (e.g., "Cloud Migration", "Data Analytics"). The `techRequirement` and `serviceRequirement` fields capture specific technical and service needs stated in the RFP/RFI. The `matchingCapability` field stores the ID of the best-matching CapabilityAsset from the knowledge base, creating a direct link from opportunity to organizational capability.

8.2 Capability Knowledge Base

The CapabilityAsset model serves as the organizational capability knowledge base. Each asset includes structured fields for solution name, accelerator, primary technology, target industry, business problem, customer outcome, differentiator, case study references, proof points, and keywords. The keywords field (JSON array) is used by the signal-capability matching engine for Jaccard similarity computation. Assets are categorized by type (service_line, solution, accelerator, technology, case_study, proof_point, objection_response, cta) and can be linked in parent-child relationships. Content hashing enables deduplication, and upvote/downvote/usage counters track effectiveness over time. This knowledge base is the foundation for both RFP/RFI matching and general sales intelligence generation.

9. Human-Controlled Selling Architecture

The platform enforces human-controlled selling through a multi-layered architectural design. The core principle is that the AI system researches, recommends, and drafts, but a human operator must explicitly approve any customer-facing communication before it is sent. This design is implemented through three complementary mechanisms: no autonomous triggers, a mandatory approval workflow, and a worker-only-send pattern.

9.1 No Autonomous Triggers

The `vercel.json` file is empty, containing zero cron job definitions. This means no background process runs on any schedule without explicit human initiation. The only cron-related route is `/api/cron/job-processor`, which is designed for manual triggering only. The email worker (`email-worker.ts`) does not run autonomously; it must be invoked by a human operator (e.g., clicking "Send Pending" in the UI), and it only processes items already in the SendQueue with status "pending" or "scheduled" with a past timestamp.

9.2 Mandatory Approval Workflow

The standard email generation flow creates a Draft record with status "pending_review". The human operator reviews the draft, optionally edits it, and either approves (status changes to "approved", creating a SendQueue entry) or rejects (status changes to "rejected" with a reason). Only approved drafts enter the SendQueue. The email worker reads exclusively from the SendQueue, never from the Draft table directly. This two-table pattern (Draft for composition, SendQueue for execution) provides a clean separation between the creative and operational phases of outbound communication.

9.3 Design Tension: sequences__process.ts

The sequence processing endpoint (`sequences__process.ts`) presents a known design tension. When processing due sequence enrollments, it creates drafts with `status: "approved"` directly (line 109) and immediately creates SendQueue entries. This bypasses the human review step for sequence-driven emails. Currently, no cron job calls this endpoint, so this code path is not active. However, if a cron job or external scheduler were configured to call this endpoint, it would enable autonomous email sending without human review. This is flagged as a mandatory Phase 4 architectural control that must be resolved before any sequence automation is activated.

10. Known Technical Debt

The following items constitute the known technical debt at the Phase 3 freeze boundary. Each item has been assessed for severity and is assigned to a future phase for resolution. No critical items remain unresolved in Phase 3; the governance layer is fully operational and all AI generation paths are covered.

ID	Item	Severity	Assigned Phase	Description
TD-01	sequences__process.ts auto-approval	High	Phase 4	Auto-sets draft status to "approved", bypassing human review. Must enforce human approval gate before any sequence automation.
TD-02	Deprecated functions in zai-helpers.ts	Low	Phase 4	callChatLLM, researchCompany, findKeyPeople, getCompanyNews, getZAI are marked @deprecated but still exported. Remove or isolate.
TD-03	No build-time governance guard	Medium	Phase 4	No automated CI/CD check prevents future files from importing callLLM directly. Add lint rule or pre-build script.
TD-04	email-generation.ts uses enforceGovernance: false	Low	Phase 5	Email generation bypasses governance enforcement. Acceptable for Phase 3 but should enforce in Phase 5 with user-facing warnings.
TD-05	No cron infrastructure	Info	Phase 4+	vercel.json has zero cron entries. Deliberate design choice for Phase 3 human-control, but limits automation capabilities.

ID	Item	Severity	Assigned Phase	Description
TD-06	No end-to-end test with live data	Medium	Phase 4	Research pipeline validated by code audit only. A live company test with full evidence chain is needed.

Table 7: Known Technical Debt Inventory

11. Phase 4/5/6 Dependency Rules

The following rules govern the dependency relationships between Phase 3 (frozen) and future phases. Phase 3 code and schema are now locked: changes require explicit justification and migration planning. Future phases must build on top of the Phase 3 architecture without modifying its core contracts.

11.1 Phase 3 Lock Rules

- The ai-governance.ts interface (governedAICall, governedAICallAggregate, recordGeneration) MUST NOT change signatures without a migration plan.
- The AIGenerationAudit schema MUST NOT remove or rename existing fields. New fields MAY be added with nullable defaults.
- The Evidence and SignalCapabilityMatch models MUST NOT change their index structure.
- The ResearchContext type from intelligence-contract.ts MUST remain backward-compatible.
- The confidence threshold values in GOVERNANCE_CONFIGS MAY be adjusted but MUST NOT be removed.
- The 15 HALLUCINATION_PREVENTION_RULES MAY be extended but existing rules MUST NOT be removed or weakened.

11.2 Phase 4 Requirements

Phase 4 must resolve the sequences__process.ts human approval bypass (TD-01) before activating any sequence automation. The architectural control is absolute: no cron, worker, scheduler, or automation mechanism may ever bypass human review for customer communication. This rule must be enforced at the code level (approval gate in sequences__process.ts), at the infrastructure level (vercel.json cron restrictions), and at the process level (documentation and team training). Phase 4 should also remove or isolate deprecated functions in zai-helpers.ts (TD-02), add a build-time governance guard (TD-03), and run a live end-to-end company test (TD-06).

11.3 Phase 5/6 Guidelines

Phase 5 should address the email-generation.ts enforceGovernance relaxation (TD-04), potentially adding user-facing warnings when generating emails with low research confidence. Phase 6 may introduce cron-based automation for non-customer-facing tasks (data enrichment, signal monitoring) but must maintain the Phase 4

human approval control for all customer communication paths. Any new AI generation types must be registered in GOVERNANCE_CONFIGS with appropriate thresholds before deployment. The build-time governance guard from Phase 4 must be extended to cover any new AI SDK integrations.